PYTHON PROGRAMMING ACTIVITY - BASED LEARNING
10 Case Studies with Solutions

• **Name:** Gudela.Yashwanth kumar

• **Roll Number:** 26000906

• **Branch:** Computer Science & Engineering (AI & ML)

• **Section:** CSM1

• **Course:** Python Programming

• **Institution:** Dr. RVR NRI Institute of Technology

Contents : -

Case Study 1: ATM Transaction Sequence Simulation (ATM trsaction.py)

Case Study 2: Mobile Plan Data Validity & Cost Optimization (data validity.py)

Case Study 3: Progressive Electricity Billing System (elctricity bill.py)

Case Study 4: Food Waste Monitoring System (food.py)

Case Study 5: Fuel Consumption and Cost Comparison (fuel consumption.py)

Case Study 6: Hospital Billing and Patient Classification (hospital bill.py)

Case Study 7: Smart Parking Fee Management (smart parkingfee.py)

Case Study 8: University Scholarship Allocation (student Scholarship.py)

Case Study 9: Attendance Recovery Computation (student attendence.py)

Case Study 10: Traffic Violation Penalty Calculator (traffic rules.py)

CASE STUDY 1: ATM Transaction Sequence Simulation

1. Problem Statement

Automated Teller Machines (ATMs) require an automated session management system to process sequential deposit and withdrawal transactions while checking withdrawal limits of Rs. 3000, enforcing available balance constraints, tracking rejected operations, and reporting account closing totals upon session exit.

2. Objectives

- Initialize account opening balance to Rs. 5000 and maintain cumulative deposit/withdrawal counters.
- Process customer transactions iteratively through a continuous while loop interface.
- Verify that withdrawal requests do not exceed the per-transaction cap of Rs. 3000 or the current account balance.
- Track and report successful deposits, withdrawals, rejected attempts, and the final account balance.

3. Assumptions

- Opening balance is set to a baseline of Rs. 5000.
- Deposit amounts must be strictly positive integers.
- Maximum withdrawal per single transaction is capped at Rs. 3000.
- Entering choice 0 exits the transaction loop and generates a transaction summary.

4. Algorithm

Start.

Initialize balance = 5000, deposits = 0, withdrawals = 0, and rejected = 0.

Display options: 1. Deposit, 2. Withdraw, 0. Exit, and input user choice.

If choice is 1, read deposit amount; if amount > 0, update balance and deposits counter; else increment rejected.

If choice is 2, read withdrawal amount; if amount <= 0 or amount > 3000 or amount > balance, reject transaction and increment rejected; else deduct from balance and add to withdrawals counter.

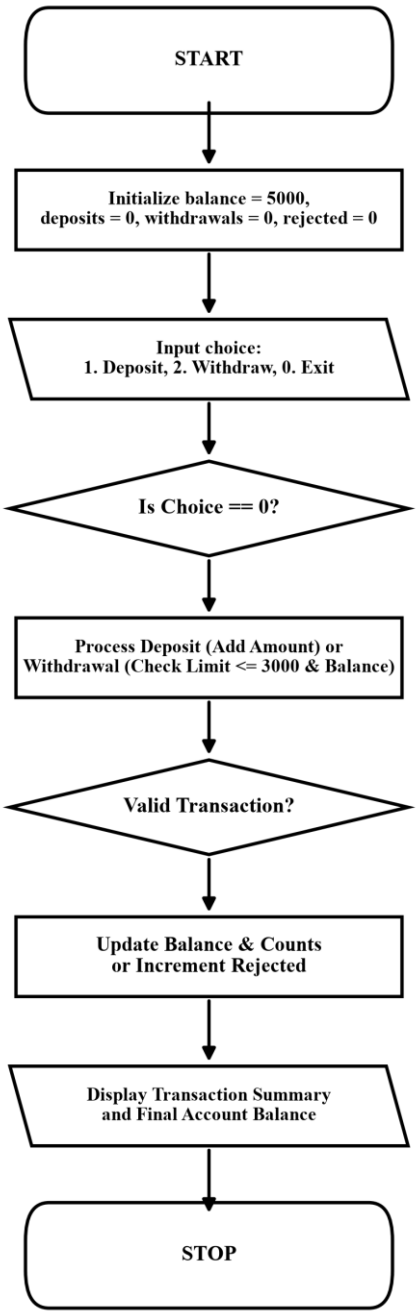
If choice is 0, terminate the interactive loop.

If choice is invalid, increment rejected counter.

Display total deposits, total withdrawals, rejected count, and closing balance.

Stop.

5. Flowchart



6. Pseudocode

```
START
```

```
    balance = 5000
    deposits = 0, withdrawals = 0, rejected = 0
    WHILE TRUE DO
        INPUT choice
        IF choice == 1 THEN
            INPUT amount
            IF amount > 0 THEN balance = balance + amount, deposits = deposits +
amount
            ELSE rejected = rejected + 1
        ELSE IF choice == 2 THEN
            INPUT amount
            IF amount <= 0 OR amount > 3000 OR amount > balance THEN rejected =
rejected + 1
            ELSE balance = balance - amount, withdrawals = withdrawals + amount
        ELSE IF choice == 0 THEN BREAK
        ELSE rejected = rejected + 1
    END WHILE
    PRINT deposits, withdrawals, rejected, balance
STOP
```

7. Python Source Code (ATM trsaction.py)

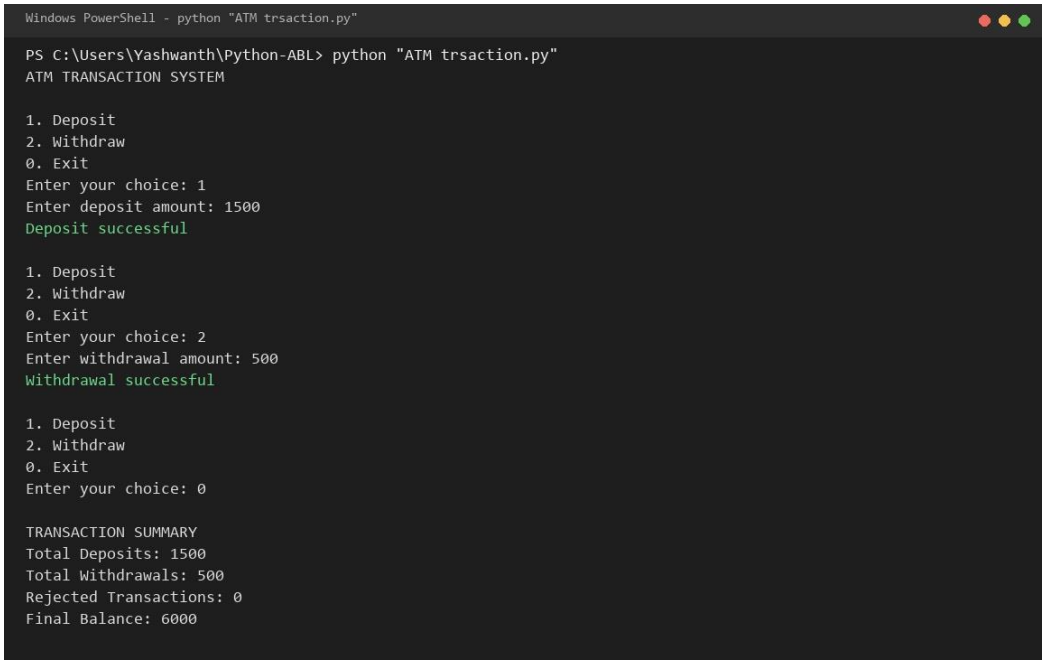
```
print("ATM TRANSACTION SYSTEM")
balance = 5000
deposits = 0
withdrawals = 0
rejected = 0
while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("0. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        amount = int(input("Enter deposit amount: "))
        if amount > 0:
            balance = balance + amount
            deposits = deposits + amount
            print("Deposit successful")
        else:
            print("Invalid amount")
            rejected = rejected + 1
    elif choice == 2:
        amount = int(input("Enter withdrawal amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif amount > 3000:
            print("Withdrawal limit exceeded")
```

```
            rejected = rejected + 1
        elif amount > balance:
            print("Insufficient balance")
            rejected = rejected + 1
        else:
            balance = balance - amount
            withdrawals = withdrawals + amount
            print("Withdrawal successful")
    elif choice == 0:
        break
    else:
        print("Invalid choice")
        rejected = rejected + 1
print("\nTRANSACTION SUMMARY")
print("Total Deposits:", deposits)
print("Total Withdrawals:", withdrawals)
print("Rejected Transactions:", rejected)
print("Final Balance:", balance)
```

8. Test Cases

Input / Situation	**Expected Result**	**Purpose**
Deposit: 1500, Withdraw: 500, Exit: 0	Deposits: 1500, Withdrawals: 500, Rejected: 0, Final: 6000	Normal sequential banking operations
Withdraw: 3500 from 5000	Rejection: Withdrawal limit exceeded, Rejected count: 1	Single transaction ceiling cap check
Withdraw: 6000 from 5000	Rejection: Insufficient balance, Rejected count: 1	Account solvency protection verification

9. Output screenshots



10. Conclusion

The ATM transaction program implements a robust state machine using while loops and conditional guards, ensuring account integrity and strict withdrawal ceiling compliance.

CASE STUDY 2: Mobile Plan Data Validity & Cost Optimization

1. Problem Statement

Mobile subscribers frequently struggle to select the most economical cellular recharge plan that satisfies their daily high-speed internet data consumption, required validity duration, and financial spending budget across competing operator tiers.

2. Objectives

- Read subscriber daily data usage requirement (GB), validity days, and maximum budget constraint.
- Model subscription specifications for Plan A (Rs.199, 1GB/day, 28d), Plan B (Rs.299, 1.5GB/day, 56d), and Plan C (Rs.399, 2GB/day, 84d).
- Evaluate eligibility conditions across all three subscription plans against user criteria.
- Recommend the most cost-effective feasible cellular plan within budget limits.

3. Assumptions

- Daily data requirements and validity days are non-negative numerical inputs.
- Subscribers require uninterrupted daily allocation meeting or exceeding usage.
- Among eligible compliant plans, the lowest subscription price is selected.

4. Algorithm

Start.

Input subscriber daily data usage (GB), required validity days, and spending budget (Rs.).

Define plan parameters: Plan A (199, 1GB, 28d), Plan B (299, 1.5GB, 56d), Plan C (399, 2GB, 84d).

Initialize best_price = 0 and best_plan = ''.

Check Plan A eligibility: if data1 >= usage and validity1 >= days and price1 <= budget, set best_plan = 'Plan A', best_price = price1.

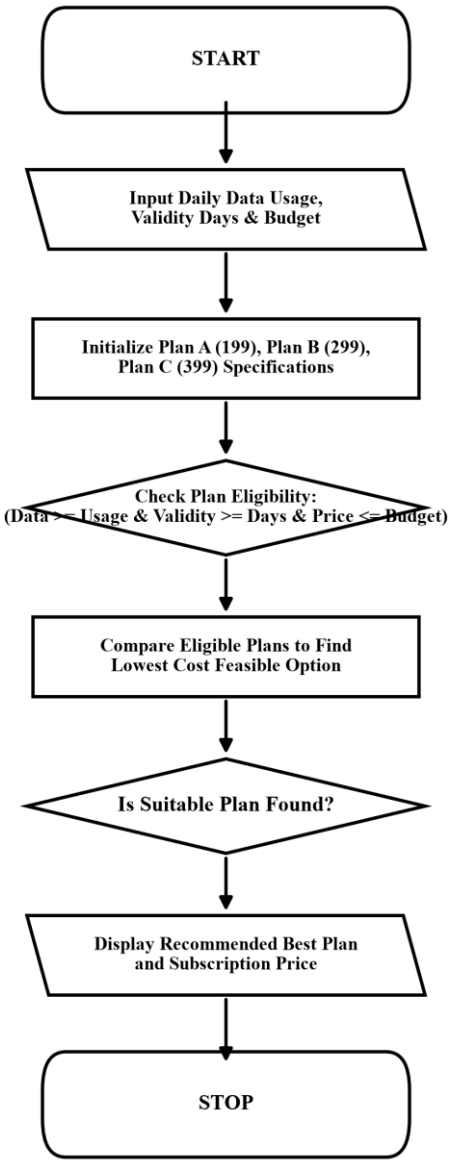
Check Plan B eligibility: if criteria satisfied and (best_price == 0 or price2 < best_price), select Plan B.

Check Plan C eligibility: if criteria satisfied and (best_price == 0 or price3 < best_price), select Plan C.

If an eligible plan is identified, display best plan name and price; otherwise report no suitable plan available.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT usage, days, budget
    best_price = 0, best_plan = ""
    IF 1.0 >= usage AND 28 >= days AND 199 <= budget THEN
```

```
        best_price = 199, best_plan = "Plan A"
    END IF
    IF 1.5 >= usage AND 56 >= days AND 299 <= budget THEN
        IF best_price == 0 OR 299 < best_price THEN best_price = 299, best_plan =
"Plan B"
    END IF
    IF 2.0 >= usage AND 84 >= days AND 399 <= budget THEN
        IF best_price == 0 OR 399 < best_price THEN best_price = 399, best_plan =
"Plan C"
    END IF
    IF best_price > 0 THEN PRINT best_plan, best_price
    ELSE PRINT "No suitable plan available"
STOP
```

7. Python Source Code (data validity.py)

```
print("MOBILE PLAN COMPARISON")

usage = float(input("Enter your daily data usage: "))
days = int(input("Enter required validity days: "))
budget = int(input("Enter your budget: "))

price1 = 199
data1 = 1
validity1 = 28

price2 = 299
data2 = 1.5
validity2 = 56

price3 = 399
data3 = 2
validity3 = 84

best_price = 0
best_plan = ""

if data1 >= usage and validity1 >= days and price1 <= budget:
    best_price = price1
    best_plan = "Plan A"

if data2 >= usage and validity2 >= days and price2 <= budget:
    if best_price == 0 or price2 < best_price:
        best_price = price2
        best_plan = "Plan B"

if data3 >= usage and validity3 >= days and price3 <= budget:
    if best_price == 0 or price3 < best_price:
```

```
        best_price = price3
        best_plan = "Plan C"

if best_price > 0:
    print("Best Plan:", best_plan)
    print("Plan Price:", best_price)
else:
    print("No suitable plan available")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Usage: 1.2 GB, Days: 28, Budget: 300	Best Plan: Plan B, Plan Price: 299	Intermediate data requirement tier selection
Usage: 0.8 GB, Days: 25, Budget: 200	Best Plan: Plan A, Plan Price: 199	Budget-conscious baseline tier selection
Usage: 3.5 GB, Days: 30, Budget: 500	No suitable plan available	Excessive daily quota exceeding all plans

9. Output screenshots



10. Conclusion

The mobile plan validation system optimizes telecom subscription costs by screening multi-attribute constraints and selecting the most economical tariff.

CASE STUDY 3: Progressive Electricity Billing System

1. Problem Statement

An electric utility provider charges consumers using tiered progressive energy slabs, applies fixed connection fees, adds high-consumption surcharges, grants prompt conservation rebates for low usage, and conducts a 3-month trend audit to detect anomalous surges.

2. Objectives

- Read monthly electricity units consumed across three consecutive billing cycles.
- Compute progressive slab energy charges (0-100 @ Rs.2, 101-200 @ Rs.3, >200 @ Rs.5).
- Add fixed connection fees (Rs. 50), compute 5% surcharge, and apply 10% low-usage conservation rebate.
- Compare bills across quarters and flag unusual consumption surges exceeding 1.5x previous periods.

3. Assumptions

- Monthly consumption units are non-negative integer readings.
- Fixed service charge of Rs. 50 applies to each monthly billing statement.
- Prompt conservation rebate of 10% is awarded only when monthly usage <= 100 units.
- A billing increase of more than 50% (> 1.5x) triggers an anomaly notification.

4. Algorithm

Start.

Input units consumed for Month 1, Month 2, and Month 3 into a list.

Iterate through months 1 to 3 using a for loop.

Compute base energy charge: if units <= 100, units * 2; elif units <= 200, 200 + (units - 100) * 3; else 500 + (units - 200) * 5.

Add fixed charge of Rs. 50. Calculate 5% surcharge and add to bill.

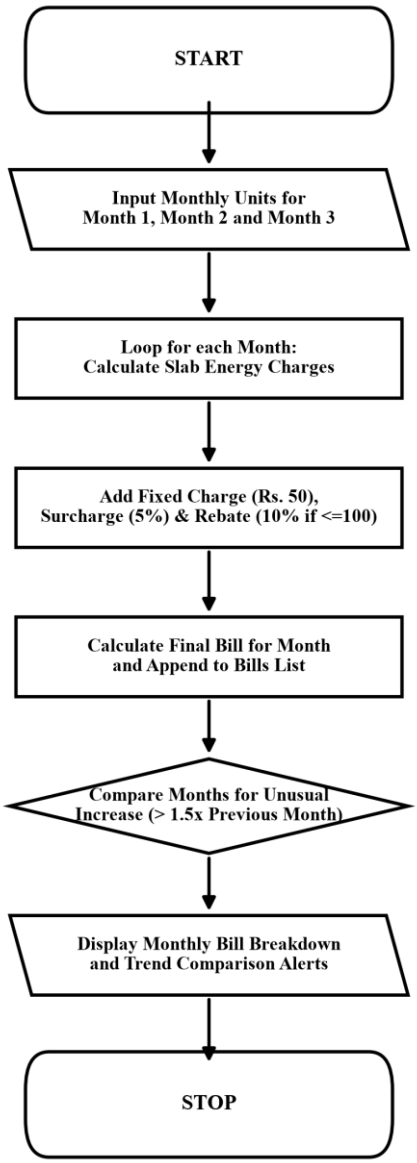
If units <= 100, calculate 10% conservation rebate and deduct from bill; else rebate = 0.

Append calculated final bill to bills list.

Display monthly bill statement and compare: if bill[1] > 1.5 * bill[0], flag Month 2 surge; if bill[2] > 1.5 * bill[1], flag Month 3 surge.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT units1, units2, units3
    units = [units1, units2, units3], bills = []
    FOR i = 0 TO 2 DO
        IF units[i] <= 100 THEN bill = units[i] * 2
        ELSE IF units[i] <= 200 THEN bill = 200 + (units[i] - 100) * 3
```

```
        ELSE bill = 500 + (units[i] - 200) * 5
        bill = bill + 50
        surcharge = bill * 0.05
        bill = bill + surcharge
        IF units[i] <= 100 THEN rebate = bill * 0.10 ELSE rebate = 0
        bill = bill - rebate
        bills.APPEND(bill)
    END FOR
    IF bills[1] > bills[0] * 1.5 THEN PRINT "Unusual increase in Month 2"
    IF bills[2] > bills[1] * 1.5 THEN PRINT "Unusual increase in Month 3"
STOP
```

7. Python Source Code (elctricity bill.py)

```
# Electricity Billing System

units1 = int(input("Enter Month 1 units: "))
units2 = int(input("Enter Month 2 units: "))
units3 = int(input("Enter Month 3 units: "))
units = [units1, units2, units3]
bills = []

for i in range(3):
    print("\nMonth", i + 1)
    print("Units:", units[i])

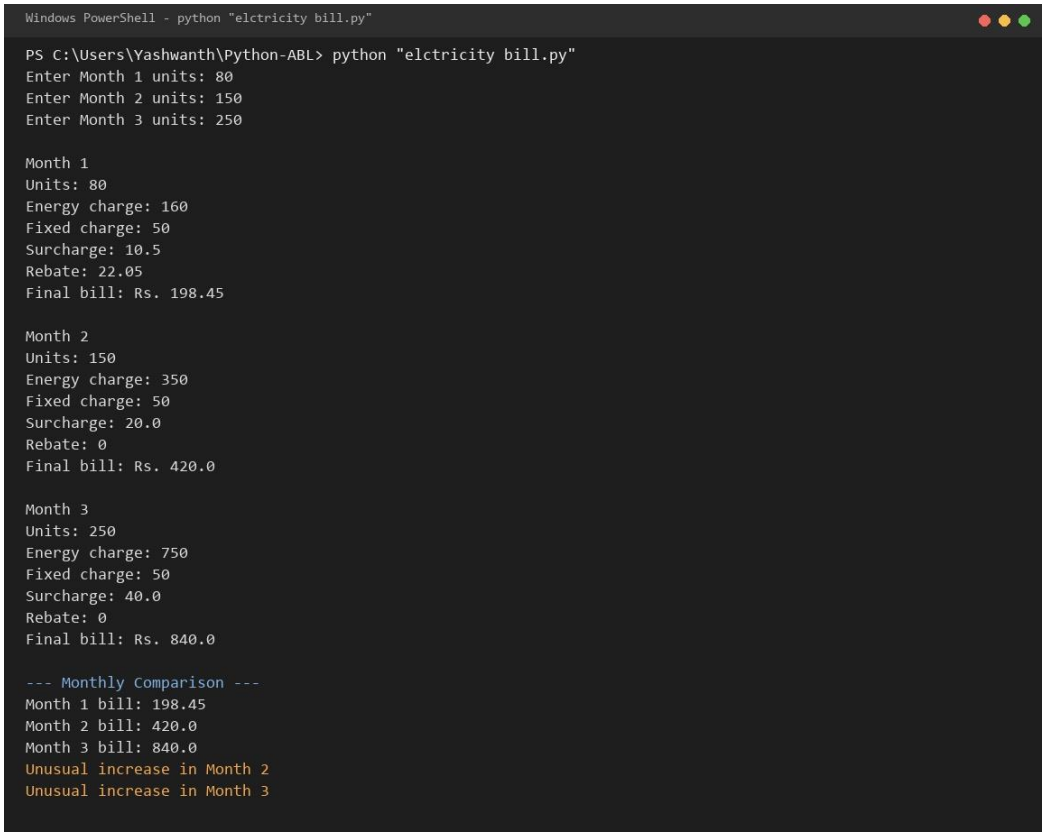
    if units[i] <= 100:
        bill = units[i] * 2
    elif units[i] <= 200:
        bill = 200 + (units[i] - 100) * 3
    else:
        bill = 500 + (units[i] - 200) * 5
    print("Energy charge:", bill)
    bill = bill + 50
    print("Fixed charge: 50")
    surcharge = bill * 0.05
    bill = bill + surcharge
    print("Surcharge:", round(surcharge, 2))
    if units[i] <= 100:
        rebate = bill * 0.10
    else:
        rebate = 0
    bill = round(bill - rebate, 2)
    print("Rebate:", round(rebate, 2))
    print("Final bill: Rs.", bill)
    bills.append(bill)
print("\n--- Monthly Comparison ---")
print("Month 1 bill:", bills[0])
```

```
print("Month 2 bill:", bills[1])
print("Month 3 bill:", bills[2])
if bills[1] > bills[0] * 1.5:
    print("Unusual increase in Month 2")
if bills[2] > bills[1] * 1.5:
    print("Unusual increase in Month 3")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Month 1: 80, Month 2: 150, Month 3: 250	M1: Rs. 198.45, M2: Rs. 420.0, M3: Rs. 840.0; Surge: M2 and M3	Progressive slab transition and surge alert
Month 1: 60, Month 2: 70, Month 3: 80	All months eligible for 10% rebate; No surge warnings	Consistent baseline low-consumption rebate
Month 1: 100, Month 2: 100, Month 3: 100	Bills identical (Rs. 236.25); Stable trend line	Steady quarterly consumption stability

9. Output screenshots



10. Conclusion

The progressive electricity billing system automates utility accounting, providing transparent tariff tiering and proactive quarterly surge detection.

CASE STUDY 4: Food Waste Monitoring System

1. Problem Statement

Institutional cafeterias and hospitality dining halls require automated food waste tracking to monitor meals prepared versus meals served, calculate leftover percentages, and issue immediate warnings when wastage exceeds acceptable thresholds.

2. Objectives

- Read total number of operational days monitored in the assessment window.
- Iterate across daily dining cycles and read prepared meal and served meal quantities.
- Compute daily leftover food and percentage wastage = (leftover / prepared) * 100.
- Evaluate wastage against the statutory 10% limit and issue high wastage alert warnings.

3. Assumptions

- Prepared meal counts are strictly positive integers.
- Served meal counts are non-negative and do not exceed prepared counts.
- The acceptable statutory threshold for food wastage is capped at 10%.

4. Algorithm

Start.

Input number of monitoring days.

Set allowable wastage limit = 10%.

For each day from 1 to days:

a. Input food prepared and food served.

b. Calculate leftover = prepared - served.

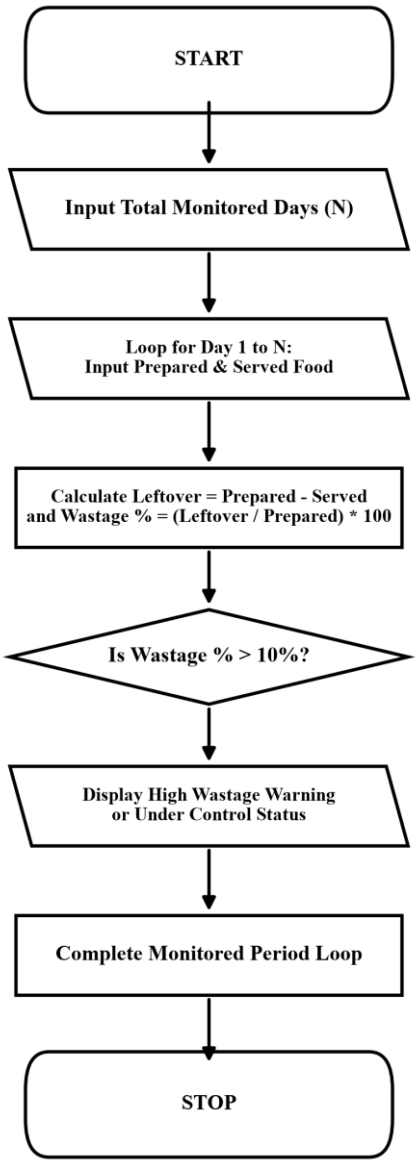
c. Calculate wastage percentage = (leftover / prepared) * 100.

d. Display leftover quantity and wastage percentage.

e. If wastage percentage > limit, print 'High food wastage!'; else print 'Food wastage is under control.'.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT days
    limit = 10
```

```
    FOR i = 1 TO days DO
        INPUT prepared, served
        leftover = prepared - served
        wastage = (leftover / prepared) * 100
        PRINT leftover, wastage
        IF wastage > limit THEN
            PRINT "High food wastage!"
        ELSE
            PRINT "Food wastage is under control."
        END IF
    END FOR
STOP
```

7. Python Source Code (food.py)

```
print("FOOD WASTE MONITORING SYSTEM")

days = int(input("Enter number of days: "))
limit = 10

for i in range(days):

    print("\nDay", i + 1)

    prepared = int(input("Enter food prepared: "))
    served = int(input("Enter food served: "))
    leftover = prepared - served
    wastage = (leftover / prepared) * 100

    print("Leftover food:", leftover)
    print("Food wastage:", wastage, "%")

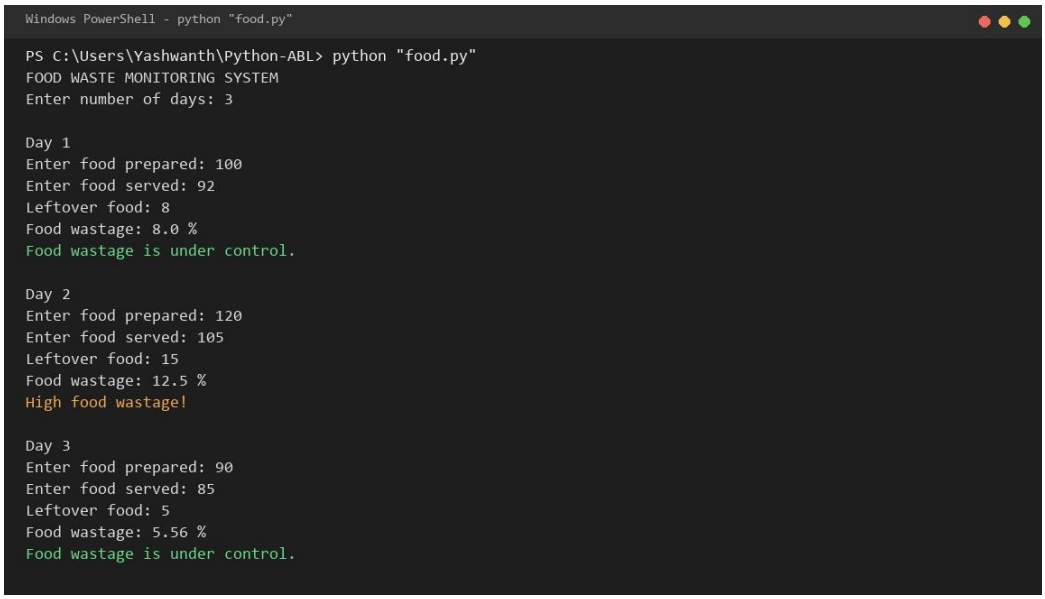
    if wastage > limit:
        print("High food wastage!")
    else:
        print("Food wastage is under control.")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Day 1: 100 prep, 92 serv; Day 2: 120 prep, 105 serv	Day 1: 8.0% (Under control), Day 2: 12.5% (High wastage warning)	Threshold breach identification
Day 1: 150 prep, 150 serv	Leftover: 0, Wastage: 0.0% (Under control)	Zero wastage optimal efficiency check

Day 1: 80 prep, 60 serv	Leftover: 20, Wastage: 25.0% (High wastage warning)	Severe wastage detection test

9. Output screenshots



10. Conclusion

The food waste monitoring system enables dining halls to identify operational food inefficiencies, supporting resource sustainability through loop-based daily audits.

CASE STUDY 5: Fuel Consumption and Cost Comparison

1. Problem Statement

Travelers and transportation fleet operators need an automated cost comparison tool to determine the most economical vehicle choice across two-wheelers, sedans, and SUVs for a designated trip distance, accounting for fuel price and safety reserves.

2. Objectives

- Read journey trip distance in kilometers from user input.
- Model vehicle mileage profiles: Bike (40 km/l), Car (20 km/l), and SUV (12 km/l).
- Incorporate a 2-liter mandatory safety reserve fuel buffer and price of Rs. 105 per liter.
- Compute total trip costs across all vehicle classes and recommend the cheapest travel alternative.

3. Assumptions

- Trip distance is a positive floating-point numerical value.
- Fuel price is fixed at Rs. 105 per liter with a 2-liter reserve margin across all vehicles.
- Vehicle fuel ratings remain constant over the duration of the trip.

4. Algorithm

Start.

Input journey distance in km.

Set fuel price = Rs. 105/liter and reserve = 2 liters.

Define fuel mileage ratings: Bike = 40 km/l, Car = 20 km/l, SUV = 12 km/l.

Compute total fuel and cost for Bike: fuel = (dist / 40) + 2, cost = fuel * 105.

Compute total fuel and cost for Car: fuel = (dist / 20) + 2, cost = fuel * 105.

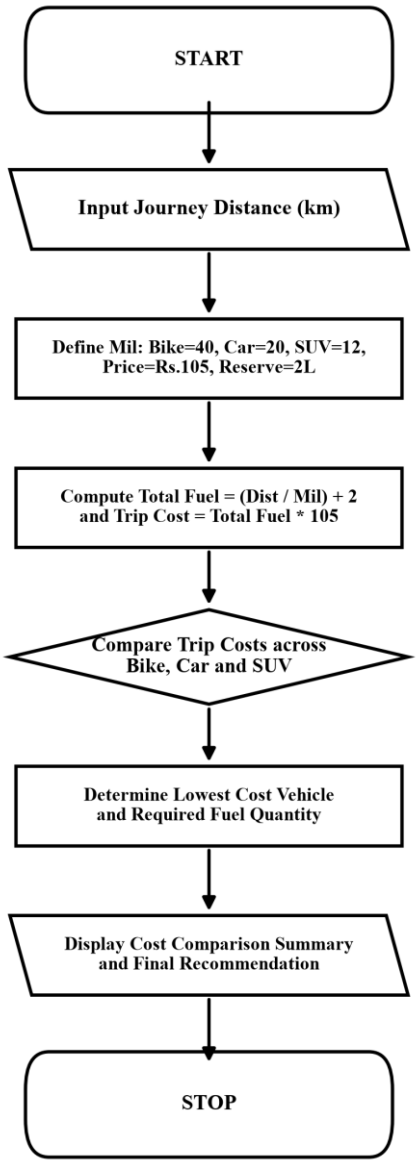
Compute total fuel and cost for SUV: fuel = (dist / 12) + 2, cost = fuel * 105.

Compare computed costs and identify lowest-cost vehicle class.

Display cost summary, cheapest vehicle recommendation, and required fuel volume.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT distance
    price = 105, reserve = 2
    bike_total = (distance / 40) + reserve; bike_cost = bike_total * price
    car_total = (distance / 20) + reserve; car_cost = car_total * price
    suv_total = (distance / 12) + reserve; suv_cost = suv_total * price
```

```
    PRINT bike_cost, car_cost, suv_cost
    IF bike_cost <= car_cost AND bike_cost <= suv_cost THEN
        PRINT "Lowest Cost Vehicle: Bike", bike_total
    ELSE IF car_cost <= suv_cost THEN
        PRINT "Lowest Cost Vehicle: Car", car_total
    ELSE
        PRINT "Lowest Cost Vehicle: SUV", suv_total
    END IF
STOP
```

7. Python Source Code (fuel consumption.py)

```
print("FUEL COST COMPARISON")
distance = float(input("Enter journey distance in km: "))
price = 105
reserve = 2

bike = 40
car = 20
suv = 12


bike_fuel = distance / bike
bike_total = bike_fuel + reserve
bike_cost = bike_total * price


car_fuel = distance / car
car_total = car_fuel + reserve
car_cost = car_total * price

suv_fuel = distance / suv
suv_total = suv_fuel + reserve
suv_cost = suv_total * price

print("Bike Cost:", bike_cost)
print("Car Cost:", car_cost)
print("SUV Cost:", suv_cost)
if bike_cost <= car_cost and bike_cost <= suv_cost:
    print("Lowest Cost Vehicle: Bike")
    print("Total Fuel Required:", bike_total)

elif car_cost <= suv_cost:
    print("Lowest Cost Vehicle: Car")
    print("Total Fuel Required:", car_total)

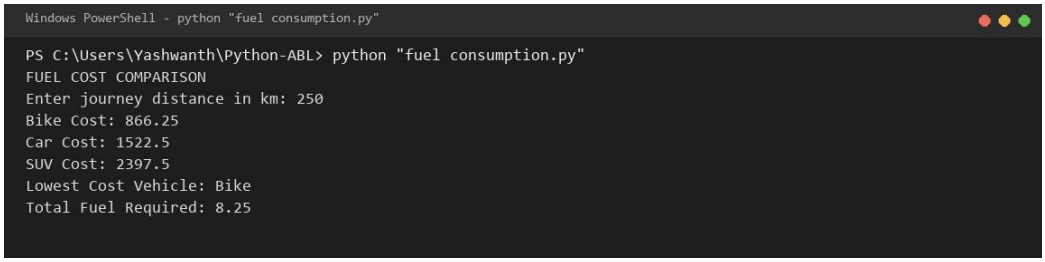
else:
    print("Lowest Cost Vehicle: SUV")
```

```
    print("Total Fuel Required:", suv_total)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Distance: 250 km	Bike: Rs. 866.25, Car: Rs. 1522.50, SUV: Rs. 2397.50 -> Lowest: Bike	Standard comparative vehicle routing
Distance: 100 km	Bike: Rs. 472.50, Car: Rs. 735.00, SUV: Rs. 1085.00 -> Lowest: Bike	Short-haul commute cost optimization
Distance: 600 km	Bike: Rs. 1785.00, Car: Rs. 3360.00, SUV: Rs. 5460.00 -> Lowest: Bike	Long-distance highway travel analysis

9. Output screenshots



10. Conclusion

The fuel cost evaluation tool provides data-driven decision making for commuters by analyzing consumption rates, safety buffers, and monetary expenses.

CASE STUDY 6: Hospital Billing and Patient Classification

1. Problem Statement

Healthcare facilities need an automated patient billing system that aggregates consultation, diagnostic, and clinical treatment fees, awards institutional discounts for bills exceeding Rs. 3000, and categorizes patients into Full Payment or Concession Payment tiers.

2. Objectives

- Read consultation, diagnostic laboratory, and clinical treatment charges.
- Calculate total hospital invoice = consultation + diagnostic + treatment.
- Apply statutory institutional discount of Rs. 500 when gross bill reaches Rs. 3000.
- Categorize patient settlement status into 'Full Payment' or 'Concession Payment'.

3. Assumptions

- All medical expense line-items are non-negative integer values.
- A flat concession of Rs. 500 is applied if aggregate charges reach or exceed Rs. 3000.
- Final invoices of Rs. 3000 or above are classified as Full Payment; otherwise Concession Payment.

4. Algorithm

Start.

Input consultation charges, diagnostic charges, and treatment charges.

Compute total bill = consultation + diagnostic + treatment.

Display total bill amount.

If total bill >= 3000, set discount = 500; else discount = 0.

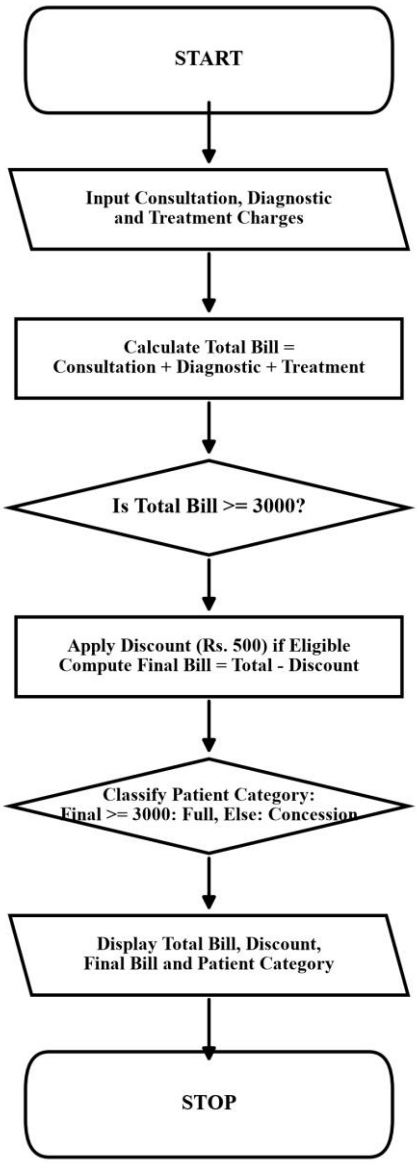
Compute final bill = total bill - discount.

If final bill >= 3000, set category = 'Full Payment'; else category = 'Concession Payment'.

Display discount applied, final payable bill, and patient classification category.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT consultation, diagnostic, treatment
    total = consultation + diagnostic + treatment
    IF total >= 3000 THEN discount = 500 ELSE discount = 0
```

```
    final_bill = total - discount
    IF final_bill >= 3000 THEN category = "Full Payment"
    ELSE category = "Concession Payment"
    PRINT discount, final_bill, category
STOP
```

7. Python Source Code (hospital bill.py)

```
print("HOSPITAL BILLING SYSTEM")

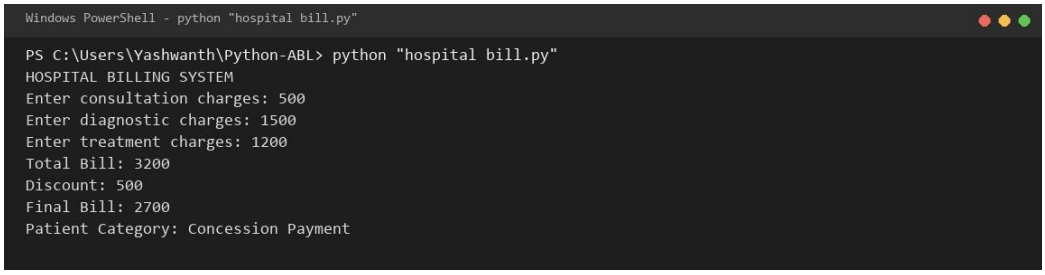
consultation = int(input("Enter consultation charges: "))
diagnostic = int(input("Enter diagnostic charges: "))
treatment = int(input("Enter treatment charges: "))

total = consultation + diagnostic + treatment
print("Total Bill:", total)
if total >= 3000:
    discount = 500
else:
    discount = 0
final_bill = total - discount
if final_bill >= 3000:
    category = "Full Payment"
else:
    category = "Concession Payment"
print("Discount:", discount)
print("Final Bill:", final_bill)
print("Patient Category:", category)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Consult: 500, Diag: 1500, Treat: 1200	Total: 3200, Discount: 500, Final: 2700, Category: Concession Payment	High total qualifying for discount and concession tier
Consult: 1000, Diag: 2000, Treat: 1500	Total: 4500, Discount: 500, Final: 4000, Category: Full Payment	Upper tier retaining Full Payment classification
Consult: 500, Diag: 800, Treat: 700	Total: 2000, Discount: 0, Final: 2000, Category: Concession Payment	Sub-threshold billing without discount

9. Output screenshots



10. Conclusion

The hospital billing system automates patient invoicing and concession management, ensuring clear accounting and automated patient tier classification.

CASE STUDY 7: Smart Parking Fee Management

1. Problem Statement

Municipal smart parking facilities require an automated billing engine that applies vehicle-specific hourly rates (Bike, Car, Bus), supports 24-hour clock rolling across midnight, enforces minimum and maximum pricing boundaries, and adds an overnight surcharge.

2. Objectives

- Read vehicle type (bike/car/bus), entry hour (0-23), and exit hour (0-23).
- Validate 24-hour clock boundaries and detect overnight parking across midnight.
- Apply category hourly rates: Bike (Rs. 10), Car (Rs. 20), Bus (Rs. 40).
- Enforce minimum fee (Rs. 20), maximum fee cap (Rs. 200), and add Rs. 50 overnight surcharge.

3. Assumptions

- Entry and exit hours are integers between 0 and 23 inclusive.
- Parking duration cannot be zero (entry != exit).
- When exit hour < entry hour, the vehicle has parked overnight across midnight.
- Minimum billing is Rs. 20 and base fee is capped at Rs. 200 before overnight surcharge.

4. Algorithm

Start.

Input vehicle type string, entry hour, and exit hour.

If entry < 0 or entry > 23 or exit < 0 or exit > 23, display 'Invalid time' and stop.

If entry == exit, display 'Invalid parking time' and stop.

Determine hourly rate: bike = 10, car = 20, bus = 40; if unknown, display 'Invalid vehicle type'.

Compute parked duration: if exit < entry, hours = (24 - entry) + exit, overnight = 1; else hours = exit - entry, overnight = 0.

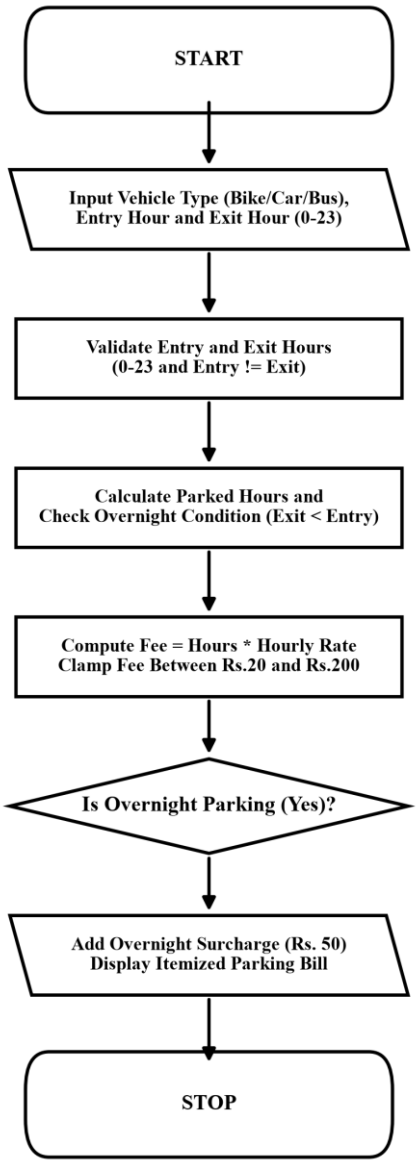
Calculate fee = hours * rate. If fee < 20, fee = 20; if fee > 200, fee = 200.

If overnight == 1, fee = fee + 50.

Display vehicle, parked hours, overnight status, and final fee.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT vehicle, entry, exit
    IF entry < 0 OR entry > 23 OR exit < 0 OR exit > 23 THEN PRINT "Invalid time"
    ELSE IF entry == exit THEN PRINT "Invalid parking time"
```

```
    ELSE
        IF vehicle == "bike" THEN rate = 10
        ELSE IF vehicle == "car" THEN rate = 20
        ELSE IF vehicle == "bus" THEN rate = 40
        ELSE PRINT "Invalid vehicle type", STOP
        IF exit < entry THEN hours = (24 - entry) + exit, overnight = 1
        ELSE hours = exit - entry, overnight = 0
        fee = hours * rate
        IF fee < 20 THEN fee = 20
        IF fee > 200 THEN fee = 200
        IF overnight == 1 THEN fee = fee + 50
        PRINT vehicle, hours, overnight, fee
    END IF
STOP
```

7. Python Source Code (smart parkingfee.py)

```
print("SMART PARKING FEE SYSTEM")
vehicle = input("Enter vehicle type (bike/car/bus): ").lower()
entry = int(input("Enter entry hour (0-23): "))
exit = int(input("Enter exit hour (0-23): "))

if entry < 0 or entry > 23 or exit < 0 or exit > 23:
    print("Invalid time")

elif entry == exit:
    print("Invalid parking time")

else:
    if vehicle == "bike":
        rate = 10
    elif vehicle == "car":
        rate = 20
    elif vehicle == "bus":
        rate = 40
    else:
        rate = 0
    if rate == 0:
        print("Invalid vehicle type")

    else:
        if exit < entry:
            hours = (24 - entry) + exit
            overnight = 1
        else:
            hours = exit - entry
            overnight = 0
```

```
        fee = hours * rate

        if fee < 20:
            fee = 20
        if fee > 200:
            fee = 200
        if overnight == 1:
            fee = fee + 50
        print("Vehicle:", vehicle)
        print("Parking Hours:", hours)
        print("Overnight:", overnight)
        print("Final Parking Fee:", fee)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Vehicle: car, Entry: 10, Exit: 16	Hours: 6, Overnight: 0, Fee: Rs. 120	Standard daytime single-day parking
Vehicle: bike, Entry: 22, Exit: 6	Hours: 8, Overnight: 1, Fee: Rs. 130 (80+50)	Midnight rollover overnight parking
Vehicle: bus, Entry: 8, Exit: 20	Hours: 12, Fee: Rs. 200 (capped from 480)	Maximum fee boundary cap enforcement

9. Output screenshots



10. Conclusion

The smart parking fee calculator handles real-world 24-hour rollover conditions and implements defensive clamping to guarantee fair municipal billing.

CASE STUDY 8: University Scholarship Allocation

1. Problem Statement

University scholarship boards require an automated applicant screening system to evaluate aggregate marks, subject-wise minimum cutoffs, attendance percentage, and family annual income to grant merit-cum-means awards.

2. Objectives

- Read applicant student name, 3 subject examination scores, attendance percentage, and annual income.
- Compute total marks (out of 300) and aggregate academic percentage.
- Verify that candidate satisfies all minimum criteria: aggregate >= 75%, each subject >= 60, attendance >= 75%, income <= Rs. 500000.
- Display comprehensive applicant academic dossier and scholarship eligibility verdict.

3. Assumptions

- Subject scores are numerical integers between 0 and 100.
- Attendance percentage is a floating-point number between 0 and 100.
- Failure to satisfy any single criterion disqualifies the applicant.

4. Algorithm

Start.

Input student name, subject 1 marks, subject 2 marks, subject 3 marks, attendance percentage, and family income.

Calculate total marks = mark1 + mark2 + mark3.

Calculate aggregate percentage = total / 3.

Display student name, subject scores, total marks, aggregate percentage, attendance, and income.

If aggregate >= 75 and mark1 >= 60 and mark2 >= 60 and mark3 >= 60 and attendance >= 75 and income <= 500000:

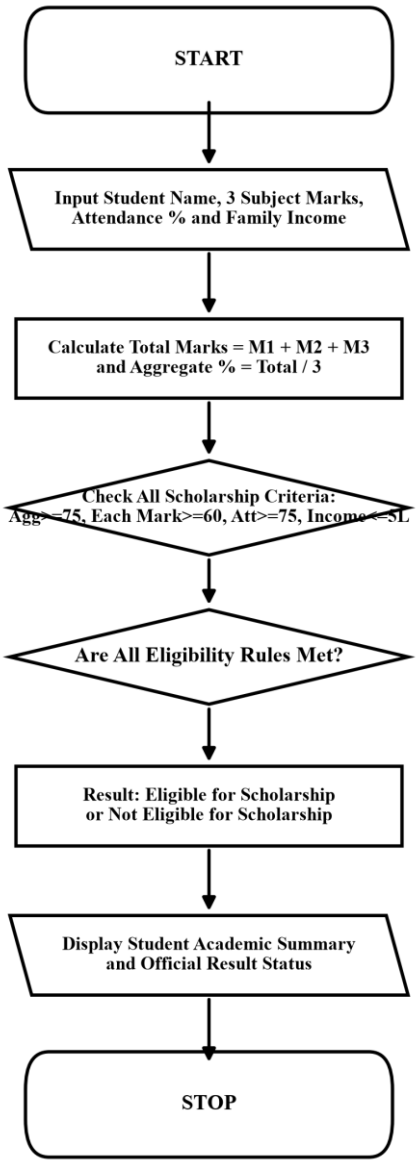
  Print 'Result: Eligible for scholarship'.

Else:

  Print 'Result: Not eligible for scholarship'.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT name, mark1, mark2, mark3, attendance, income
    total = mark1 + mark2 + mark3
    aggregate = total / 3
```

```
    PRINT name, mark1, mark2, mark3, total, aggregate, attendance, income
    IF aggregate >= 75 AND mark1 >= 60 AND mark2 >= 60 AND mark3 >= 60 AND attendance
>= 75 AND income <= 500000 THEN
        PRINT "Result: Eligible for scholarship"
    ELSE
        PRINT "Result: Not eligible for scholarship"
    END IF
STOP
```

7. Python Source Code (student Scholarship.py)

```
# student Scholarship System
name = input("Enter student name: ")
mark1 = int(input("Enter subject 1 marks: "))
mark2 = int(input("Enter subject 2 marks: "))
mark3 = int(input("Enter subject 3 marks: "))
attendance = float(input("Enter attendance percentage: "))
income = int(input("Enter family income: "))

total = mark1 + mark2 + mark3
aggregate = total / 3

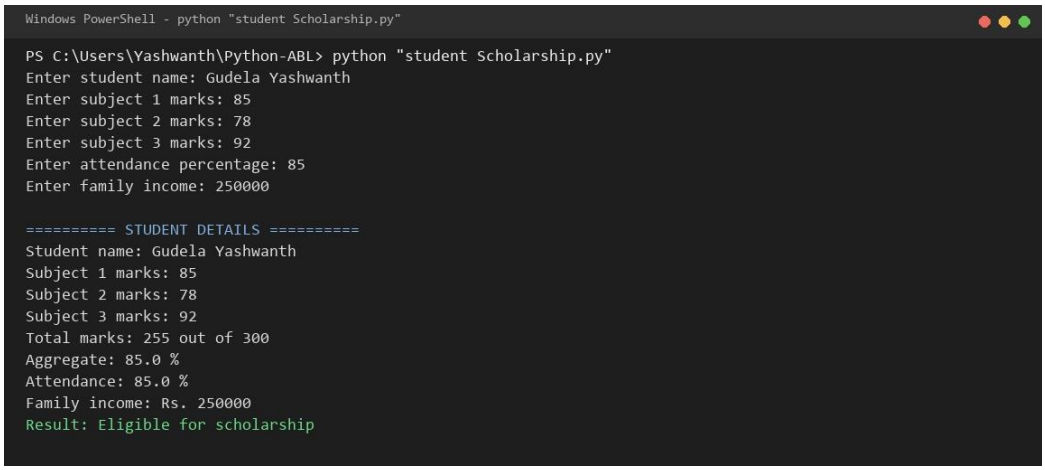
print("\n========== STUDENT DETAILS ==========")
print("Student name:", name)
print("Subject 1 marks:", mark1)
print("Subject 2 marks:", mark2)
print("Subject 3 marks:", mark3)
print("Total marks:", total, "out of 300")
print("Aggregate:", aggregate, "%")
print("Attendance:", attendance, "%")
print("Family income: Rs.", income)
if (aggregate >= 75 and mark1 >= 60 and mark2 >= 60 and mark3 >= 60 and attendance >=
75 and income <= 500000):
    print("Result: Eligible for scholarship")
else:
    print("Result: Not eligible for scholarship")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Gudela Yashwanth, Marks: 85, 78, 92, Att: 85%, Inc: 250000	Aggregate: 85.0%, Result: Eligible for scholarship	Fully compliant merit-cum-means profile
Ravi, Marks: 55, 80, 85, Att: 80%, Inc: 200000	Result: Not eligible (Subject 1 below 60 cutoff)	Individual subject minimum cutoff failure

Pooja, Marks: 90, 85, 88, Att: 70%, Inc: 300000	Result: Not eligible (Attendance below 75%)	Mandatory attendance shortfall rejection

9. Output screenshots



10. Conclusion

The scholarship screening system demonstrates compound logical condition evaluations, ensuring strict adherence to university eligibility policies.

CASE STUDY 9: Attendance Recovery Computation

1. Problem Statement

Undergraduate students with attendance shortages need an interactive calculator to determine their current attendance percentage and compute the exact minimum consecutive future lectures they must attend without absence to attain the statutory 75% examination threshold.

2. Objectives

- Read total classes conducted, classes attended, and required attendance percentage.
- Validate numerical input bounds (positive total, attended <= total, required in 0-100).
- Calculate current attendance percentage and check if already eligible.
- Use an iterative while loop to calculate the exact consecutive future classes needed.

3. Assumptions

- Total conducted classes is a positive integer greater than zero.
- Classes attended is a non-negative integer less than or equal to total classes.
- The student attends every upcoming recovery class consecutively without missing any.

4. Algorithm

Start.

Input total classes, attended classes, and required attendance percentage.

Validate input ranges: total <= 0 or attended < 0 or attended > total or required <= 0 or required > 100.

If invalid, print 'Invalid input' and stop.

Calculate current percentage = (attended / total) * 100.

Display current attendance percentage.

If current >= required, print 'You are already eligible'.

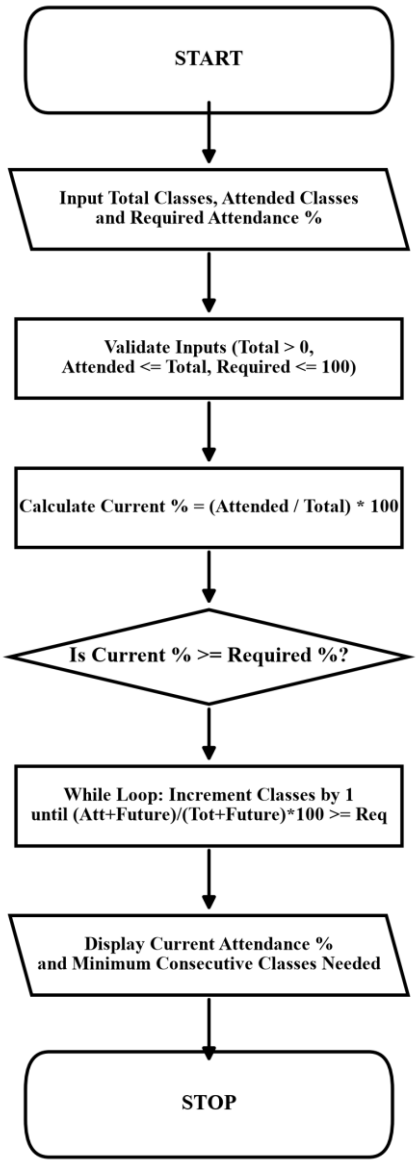
Otherwise, initialize future = 0.

While ((attended + future) / (total + future)) * 100 < required: future = future + 1.

Display minimum classes to attend.

Stop.

5. Flowchart



6. Pseudocode

```
START
    INPUT total, attended, required
    IF total <= 0 OR attended < 0 OR attended > total OR required <= 0 OR required >
100 THEN
```

```
        PRINT "Invalid input"
    ELSE
        current = (attended / total) * 100
        PRINT current
        IF current >= required THEN
            PRINT "You are already eligible"
        ELSE
            future = 0
            WHILE ((attended + future) / (total + future)) * 100 < required DO
                future = future + 1
            END WHILE
            PRINT future
        END IF
    END IF
STOP
```

7. Python Source Code (student attendence.py)

```
print("ATTENDANCE RECOVERY CALCULATOR")
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))
required = float(input("Enter required attendance percentage: "))

if total <= 0 or attended < 0 or attended > total or required <= 0 or required > 100:
    print("Invalid input")
else:
    current = (attended / total) * 100
    print("Current Attendance:", current, "%")
    if current >= required:
        print("You are already eligible")
    else:
        future = 0

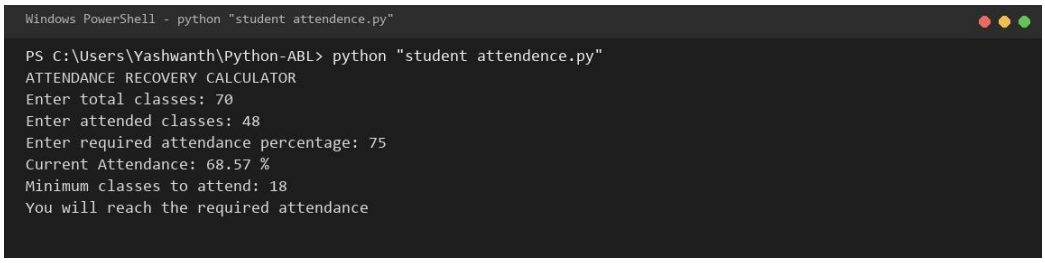
        while ((attended + future) / (total + future)) * 100 < required:
            future = future + 1
        print("Minimum classes to attend:", future)
        print("You will reach the required attendance")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Total: 70, Attended: 48, Required: 75%	Current: 68.57%, Minimum classes to attend: 18	Attendance shortage iterative recovery
Total: 100, Attended: 80, Required: 75%	Current: 80.0%, You are already eligible	Pre-qualified threshold verification

Total: 50, Attended: 60, Required: 75%	Invalid input	Boundary validation (attended > total)

9. Output screenshots



10. Conclusion

The attendance recovery tool provides precise academic remedial guidance using natural while-loop iteration to solve ratio convergence.

CASE STUDY 10: Traffic Violation Penalty Calculator

1. Problem Statement

Municipal traffic regulatory authorities require an automated e-challan violation penalty calculator that tallies statutory penalties across multiple offenses, doubles fines for repeat offenders, and flags drivers for license disqualification when cumulative infractions reach two or more.

2. Objectives

- Read number of traffic violations committed by the driver in the incident.
- Iterate across infractions and read offense category (No Helmet, No License, Using Mobile).
- Double fines for repeat offenses: Helmet (500/1000), License (1000/2000), Mobile (1500/3000).
- Calculate total fine, count violations, and trigger driver disqualification if violations >= 2.

3. Assumptions

- Violation selection options: 1: No Helmet, 2: No License, 3: Using Mobile.
- Repeat offense status is answered with 'yes' or 'no'.
- A driver committing two or more violations is flagged for driving license disqualification.

4. Algorithm

Start.

Initialize fine = 0, count = 0.

Input number of violations n.

For i from 0 to n - 1:

a. Display violation menu and input offense number.

b. Input repeat offense status ('yes' or 'no').

c. If offense == 1, fine = fine + (1000 if repeat=='yes' else 500), count = count + 1.

d. If offense == 2, fine = fine + (2000 if repeat=='yes' else 1000), count = count + 1.

e. If offense == 3, fine = fine + (3000 if repeat=='yes' else 1500), count = count + 1.

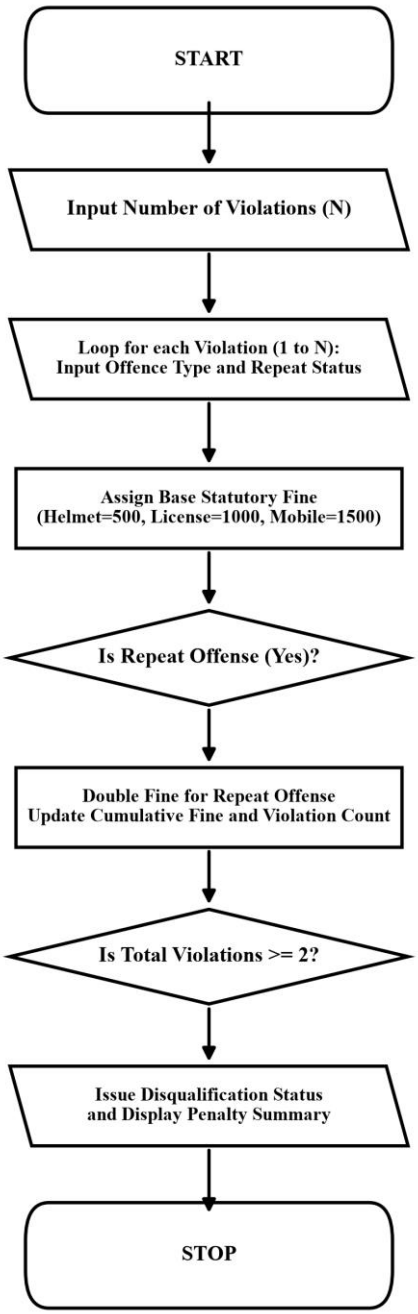
f. Else print 'Invalid offence'.

Display total fine and total violation count.

If count >= 2, print 'Disqualification: Yes'; else print 'Disqualification: No'.

Stop.

5. Flowchart


START
Input Number of Violations (N)
Loop for each Violation (1 to N):
Input Offence Type and Repeat Status
Assign Base Statutory Fine
(Helmet=500, License=1000, Mobile=1500)
Is Repeat Offense (Yes)?
Double Fine for Repeat Offense
Update Cumulative Fine and Violation Count
Is Total Violations >= 2?
Issue Disqualification Status
and Display Penalty Summary
STOP

6. Pseudocode

```
START
    fine = 0, count = 0
    INPUT n
    FOR i = 1 TO n DO
        INPUT offence, repeat
        IF offence == 1 THEN
            IF repeat == "yes" THEN fine = fine + 1000 ELSE fine = fine + 500
            count = count + 1
        ELSE IF offence == 2 THEN
            IF repeat == "yes" THEN fine = fine + 2000 ELSE fine = fine + 1000
            count = count + 1
        ELSE IF offence == 3 THEN
            IF repeat == "yes" THEN fine = fine + 3000 ELSE fine = fine + 1500
            count = count + 1
        END IF
    END FOR
    PRINT fine, count
    IF count >= 2 THEN PRINT "Disqualification: Yes" ELSE PRINT "Disqualification: No"
STOP
```

7. Python Source Code (traffic rules.py)

```
print("TRAFFIC VIOLATION PENALTY CALCULATOR")
fine = 0
count = 0
n = int(input("Enter number of violations: "))
for i in range(n):
    print("\n1. No Helmet")
    print("2. No Licence")
    print("3. Using Mobile")

    offence = int(input("Enter offence number: "))
    repeat = input("Is it a repeat offence? (yes/no): ").lower()

    if offence == 1:
        if repeat == "yes":
            fine = fine + 1000
        else:
            fine = fine + 500

        count = count + 1

    elif offence == 2:
        if repeat == "yes":
            fine = fine + 2000
        else:
            fine = fine + 1000
```

```
        count = count + 1

    elif offence == 3:
        if repeat == "yes":
            fine = fine + 3000
        else:
            fine = fine + 1500

        count = count + 1

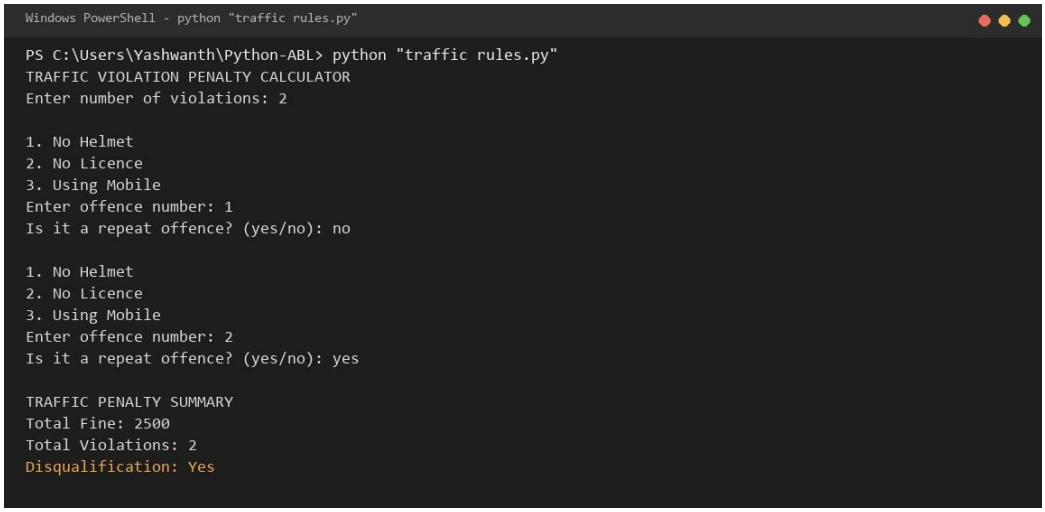
    else:
        print("Invalid offence")

print("\nTRAFFIC PENALTY SUMMARY")
print("Total Fine:", fine)
print("Total Violations:", count)
if count >= 2:
    print("Disqualification: Yes")
else:
    print("Disqualification: No")
```

8. Test Cases

Input / Situation	**Expected Result**	**Purpose**
Violations: 2; Offence 1: Helmet (no), Offence 2: License (yes)	Total Fine: 2500, Violations: 2, Disqualification: Yes	Compound multi-offense penalty check
Violations: 1; Offence 1: Helmet (no)	Total Fine: 500, Violations: 1, Disqualification: No	Single first-time infraction without disqualification
Violations: 1; Offence 3: Mobile (yes)	Total Fine: 3000, Violations: 1, Disqualification: No	Repeat mobile usage penalty without disqualification

9. Output screenshots



10. Conclusion

The traffic violation penalty calculator enforces road safety regulations through tiered multi-offense fines and automated driver disqualification tracking.